Journal of Intelligent Decision Making and Information Science

www.jidmis.org
eISSN: 3079-0875



Autonomous Security Agents in Cloud Environments: Leveraging Machine Learning and Agentic AI

Dr. Madhuri Kovoor[1], Dr. PVS Siva Prasad[2], Dr. Venkata Reddy Medikonda[3], Dr. G. Balram[4], C Dastagiraiah[5], Rajasekhar Reddy Pocha[6], B. V. V. Siva Prasad[7], Kamalakar Ramineni[8]

[1]Associate Professor, School of Engineering, Department of CSE, Anurag University, Ghatkesar, Hyderabad, madhuris97@gmail.com, https://orcid.org/0000-0001-5046-838X

[2]Associate Professor, School of Engineering, Department of CSE, Anurag University, pvssivaprasadcse@anurag.edu.in, https://orcid.org/0009-0009-8951-4748

[3]Associate Professor, Department of Computer Science and Engineering, Marri Laxman Reddy Institute of Technology and Management, venkat.medikonda@gmail.com, https://orcid.org/0009-0001-5295-9091

[4]Assistant Professor, School of Engineering, Department of CSE, Anurag University, balaramcse@anurag.edu.in Orcid: https://orcid.org/0009-0004-7167-1503

[5]Assistant Professor, School of Engineering (CSE), Anurag University, Ghatkesar, Hyderabad, dattu5052172@gmail.com, https://orcid.org/0000-0003-3749-2873

[6]Assistant Professor, School of Engineering (CSE), Anurag University, Ghatkesar, Hyderabad, rajasekharreddycse@anurag.edu.in, https://orcid.org/0000-0001-7477-0428

[7]Associate Professor, School of Engineering, Department of CSE, Anurag University, Ghatkesar, Hyderabad, drbvvsivaprasad@gmail.com, https://orcid.org/0000-0001-8650-3984

[8](Corresponding Author)Assistant Professor, School of Engineering (CSE), Anurag University, kamalsee@gmail.com, https://orcid.org/0009-0000-6074-9433

ARTICLE INFO

Article history:
Received 26 June 2026
Received in revised form 27 July 2026
Accepted 06 August 2026

ABSTRACT

Autonomous security agents in cloud environments can be organized as a three-layer pipeline in which machine-learning-based detection identifies suspicious activity, agentic AI selects investigative steps and proposes responses, and a separately governed control layer authorizes and verifies consequential actions. This study specifies that integration boundary and evaluates the response-control layer using a narrow AWS IAM role-session operation. The ML interface carries detector version, score semantics, feature provenance, and supporting events; the agentic interface carries typed evidence requests and action proposals but no execution credentials. The control contract binds admitted evidence, current resource context, exact policy generation, action-specific approval, intervention capacity, worker fencing, and uncertain-result reconciliation. Evaluation includes 32 predicate fixtures, all 2,187 assignments of seven three-valued conditions, 66 malformed-input checks, 29 composed schedules, and structural and source-level policy mutations. The full transition configurations contain 19,516 reachable states and 76,948 transitions; five specified invariants hold within those bounds. In a separate 12-schedule comparison, admission-snapshot authorization produces four invalid issues and seven invalid effects, whereas the proposed issue-time controller produces no invalid issues but retains three post-issue invalid effects; an idealized atomic effect-boundary reference prevents those effects under a stronger modeled capability. All configurations preserve the three authority-preserving completions. The comparison extends the base paper's distinction between endpoint success and

Keywords: autonomous security agents; cloud security; machine learning; agentic AI; incident response; authorization; finite-state analysis; identity and access management.

authorized completion to a concrete IAM contract; it neither reproduces that paper's agent experiments nor establishes superiority over its monitor. No cloud service, trained detector, language-model agent, or human-participant experiment was executed. The results therefore validate bounded control-layer behavior and its interfaces, not gains in detection accuracy or agent reasoning. The framework shows how ML findings and agentic investigation can be leveraged for autonomous cloud security without treating model outputs as execution authority.

1. Introduction

Cloud incident response relates observations to operations which can change access, disrupt applications, or maintain evidence. NIST SP 800-61 Revision 3 places incident handling in the context of organizational cybersecurity risk management [1]. This view allows the response decision – and not just the detection finding – to be the unit of security analysis. Machine learning and agentic AI are not mutually exclusive methods in the architecture studied here, but sequential tasks. Layer 1 uses ML-based detection for cloud telemetry and outputs a versioned finding with score semantics, feature provenance, and evidence references. Layer 2 employs agentic AI to identify allowed questions to investigate, refine alternative hypotheses, and generate a typed response proposal. A consequential cloud action can only be taken after Layer 3 has independently reviewed evidence, context, authorization, dispatch and verification. Amazon GuardDuty records suspicious-activity detection using machine learning [2] and ReAct gives a solid framework for tools that combine reasoning, actions and observations [3]. This study doesn't claim anything new in the detector or the algorithm used for agent planning; it is based on those roles. In the present study four questions are distinguished. Is there evidence to back up the incident assessment? Does the specific operation have a specific permission? Does it have expected outcomes that are acceptable given the relevant policy? Have the desired results been achieved? Answers to these questions may vary. An authorized policy update may not be suitable due to misleading evidence or a justified policy update may not be verified after timeout. Untrusted retrieved material adds to the complexity of investigations. AgentDojo assesses attacks that are made via tool-returned content and legitimate agent-task failures [4]. OWASP suggests restricting functionality and permissions and implementing authorization at the level of the language model [5]. In light of this, this work makes the assumption that the agent can make false or adversarial offers. The control layer should not rely on the agent being able to detect all malicious instructions. Governed agents are already known research subjects. Progent provides symbolic privilege enforcement [6], Agentra offers supervised security-response planning [7] and TraceGrant links contracts, admitted evidence, task state, effect authorization, and completion checks [8]. A proposal simply adding an agent, a permission check and an audit log would not be sufficiently differentiated. The basic method of this study is to use commit-time authorization research [9] as a methodological foundation, but instead focus on an implementation boundary: an authorized cloud mutation can be prepared, issued, delayed, applied, or left unknown as authorization and shared response capacity change. The research question is as follows: How to combine machine learning based threat detection and agentic AI investigation with evidence-based authorization and recoverable response to enable autonomous security action in cloud environments

and which properties of a control can be defined in a finite executable model? RQ1 seeks to understand the information and authority boundaries between ML detection, agentic investigation and autonomous response. RQ2 is to determine what control obligations ought to be made under the conditions of approval withdrawal, concurrency, failure and uncertain external effects. RQ3 is differentiating between properties that can be established by the executable control model and properties that need to be evaluated by detector, agent and cloud-provider. The contribution consists of three parts. The article introduces first the concept of a three-layer integration architecture, in which the findings of the ML become bounded agent investigations and agent proposals enter into independent control of response. Secondly, it creates that control contract for one AWS IAM response operation, specifying exact parameters, evidence predicates, approval binding, recoverable dispatch and verification limits. Third, it provides a well-formed contract-to-dispatch harness, finite transition exploration, mutation tests, and a controlled comparison of the timing of authorization. Evaluated is the response layer and agent-to-control boundary; detector training and/or benchmark agent reasoning is not evaluated. The work is not intended to introduce new access-control theory, prove a better incident detection or prove that the combination is new. Related work and method are established in sections 2 and 3. Requirements, operation contract and lifecycle are defined in sections 4 – 6. The results of the models are reported in Section 7. In sections 8-10, deployment obligations, validity limitations and conclusions are distinguished.

2. Related work and contribution boundary

2.1 Access-control and usage-control foundations

Based on attributes, the subject, object, operation and environment attributes [10] are considered. The UCONABC model also introduces authorizations, obligations, conditions, continuity and mutability to the usage control [11]. In this context, the foundations of permission and reevaluation during activity are not new concepts brought in by the agent terminology, but rather are taken up as context dependent. XACML 3.0 specifies policy decisions, indeterminate outcomes and combining algorithms, such as deny-overrides [12]. The predicates used below are local true/falsed/unknown that are simpler and don't contain XACML's extended indeterminate semantics or the full policy language. They are designed to enable inspection of the decision made by the operation. No claim to conformance with XACML is made.

2.2 Agent enforcement and response planning

Progent is able to analyze fine-grained tool policies and differentiate checked privilege narrowing from expansion. It is claimed that it cannot withstand attacks inside the legitimate privileges and manipulation of text outputs [6]. AgentSpec offers runtime rules grouped by triggers, predicates and enforcement actions [13]. These systems can be used to enforce external systems and demonstrate that tool authorization does not equal semantic correctness.

Agentra is a near neighbor security-response comparator. It creates structured plans in a supervised planning and validation process, action restrictions and audit records. It is a projected assessment of plans, rather than a measurement of live cloud effects, as is its harmful-action metric. The difference is important, because the current study also doesn't measure operational cloud harm, but rather a

different object: asynchronous transitions after an admitted response proposal. TraceGrant is the most similar lifecycle comparator. The contract specifies allowed effects, obligations, evidence bindings and completion conditions; at runtime, controls are added that bind qualified evidence and remaining authority to native tool execution [8]. This significant overlap precludes a novelty argument of combining evidence, budgets, authorization and receipts. This work provides a slim IAM operation contract and executable schedules for preparation recovery, ambiguous dispatch and shared intervention accounting. The schedules are a comparison target and not proof that TraceGrant can't be extended to handle those schedules.

2.3 Primary base paper and comparative claim

The main paper is Temporary Authority, Permanent Effects: Commit-Time Authorization for LLM Agents, arXiv:2607.10487v1, from July 2026 [9]. It separates effect from valid authority and requires evidence to be visible to make CommitGuard's protection conditional on the final check and atomic coupling of it to the effect. TraceGrant is the most similar lifecycle comparator available [8]. The current specialization explores what is still enforceable when the chosen IAM API doesn't couple an organizational approval with its eventual effect. It has other artifacts that link together policy semantics, owned prepared-workers, reserved follow-up resources, and fuzzy dispatch. This is not to say that there are not existing frameworks that can be extended, but rather a comparison of concrete obligations. Section 7 compares the results of the base paper with new schedules that were conducted. The local effect-boundary reference is an independently written idealized mechanism, and is not the original CommitGuard implementation.

2.4 Feature-level comparison

Table 1 shows the inherited mechanism and what additional question is being pursued here. The term “not established by the cited evaluation” is not used as a synonym for “unsupported by the approach,” but rather is intentionally used to denote that the approach failed to provide evidence for the item. No re-implementation or benchmarking of the named systems was done for this study.

Table 1. Prior mechanisms and the present study's narrower analysis target.

Prior approach	Established mechanism	Relationship to this study
ABAC and UCON [10,11]	Attribute-based decisions; ongoing conditions and mutable state	Foundation for contextual authority; this work instantiates response predicates and failure transitions
XACML 3.0 [12]	Explicit decision semantics and policy combination	Foundation for fail-closed treatment; local three-valued rules are not a replacement or compliant implementation
Progent [6]	Symbolic tool privileges and controlled privilege updates	Tool gating is inherited; the present model disallows agent-initiated expansion and examines external-effect uncertainty

Prior approach	Established mechanism	Relationship to this study
AgentSpec [13]	Runtime triggers, predicates, and enforcement actions	Runtime checks are inherited; concrete role-session and accounting semantics are specified here
Agentra [7]	Governed response planning and projected plan assessment	Complementary planning focus; current experiments hold proposals fixed and do not compare agent reasoning
TraceGrant [8]	Contract-bound evidence, authority, budgets, native results, and completion	Broader lifecycle comparator; no architectural priority claimed
Commit-time authorization [9]	Authorization evaluated at a protected effect boundary	Primary base paper; this work examines a fixed IAM operation without an atomic approval/effect primitive

2.5 What is and is not contributed

The incremental contribution is a contract that can be verified for a specific effect boundary. It asserts something that must be true for an operation, it leaves something undetermined when the outcome of an operation is not determined, and it allows a verifier to assert something that is legitimate after the operation. These details reveal a difference that is sometimes hidden behind general diagrams: It is not enough to maintain a correct local accounting invariant to get a correct incident judgment or currently authorized remote effect.

The literature search was focused. The comparison is thus a justifiable positioning and not a comprehensive demonstration of novelty. Completed results do not measure the accuracy of detection nor compare agent planners. Rather, the study identifies the information that needs to be transferred between the two processes of ML detection and agentic investigation, and between the two processes of agentic investigation and controlled execution, and it experimentally explores the latter boundary. The architecture of machine learning and agentic AI thus form part of the building blocks, but not their predictive and reasoning capabilities, which are not reported as empirical results of this study.

3. Method and completed research activities

3.1 Study design

This is a study of a design and a model that can be executed. It includes a specification based on literature and finite-state exploration and deterministic regression checks. Conceptual research should have an explicit design and conceptual foundations that are justified [14] and design-science methodology involves problem identification, artifact construction, demonstration and evaluation [15]. In this case, the evaluated artifact is not an operating security service, but rather a Python abstraction of some selected control rules.

The following activities were completed: source checking, requirement derivation, operation-level specification, predicates and transitions implemented, test executed, and counterexamples examined. No detector was trained, no agent endpoint was queried, no cloud credential was utilized and no participant was recruited. Statements regarding those activities are not deferred promises, or are implied by the proposed experiment.

3.2 Evidence collection and traceability

Primary research and published/authored records, standards and official provider documents were reviewed on September 1, 2026. Searches were a mix of cloud response, agent authorization, runtime control and exact close comparator titles. TraceGrant, UCON, and XACML were introduced to the comparative positioning. Authors' supplied manuscript headers that include a journal are not enough to indicate the inspected version is a preprint. Official documentation does not support comparative performance, it supports interface semantics.

A registered systematic review protocol, full database coverage, dual screening and invented screening counts were not used in the lookup. A source log and requirement-traceability file are included in the supplement. Source verification is not proof of the validity of a cited study. The base paper was examined using the versioned PDF and HTML version as well as the results tables reported in the paper, but the base paper's original executable artifact was not retrieved or executed.

3.3 Evaluation objects and oracles

Four different objects are being tested. Predicate fixtures act tests on evidence, context and approval rules, using structured inputs. A decision enumeration tests all the assignments to seven three-valued conditions against the exact permit condition. Two operations with a limited allowance are explored in a separate transition model, with approvals, context, worker preparation and provider outcomes changing. Lastly, a composed harness validates proposals and admitted facts, builds and verifies the actual policy, attaches approval to the request, reserves resources, enforces caller epochs and projects provider outcomes. This is a fourth object that makes connections between predicate and transition tests that are not addressed.

The model keeps provider state information hidden in the system for determining the results of a reconciliation, but the allowed transitions in the broker determine whether a reconciliation is considered successful or failed, or whether it is simply not dispatched. An unknown request with hidden state NONE is not automatically released, the broker has no proof that there is still an effect that can occur. This will avoid the test oracle from giving information to the broker that is not specified.

The expected fixture values and invariants were not written by an independent review panel, but rather along with the specification. The permit oracle is not an independent security check, but it is implemented separately as an all-true check. The finite model is not a guarantee of the refinement of the implementation of a future broker and has its own assumptions. These restrictions help to limit the inference of a passing test.

3.4 Reproducibility and mutation testing

The artifact includes machine-readable fixtures, a gate truth table, exploration summaries, full counterexample traces, and environment metadata, all created by run_all.py. Breadth-first traversal explores the reachable graph of each finite configuration given, it doesn't stop at some arbitrary level. The complete model, the bounds and the variant switches are available in the manuscript. A second run in a separate directory to check for deterministic result agreement; environment metadata is reported separately from scientific outcomes.

There are three deliberately weakened variants that eliminate dispatch-time revalidation, separate capacity observation from reservation or release uncertain reservations. They are not copies of commercial products or published baselines, but variations of the design that fail to meet the requirements.They are design variants that are fault-seeded, not copies of commercial products or published baselines. They are not meant to build an unfair superiority claim, but rather to find out if the analysis can determine the corresponding failure. Other mutations modify the policy document created and, if used in a disposable source document, the comparator or the cutoff. The mutation detection is not a population estimate of defect coverage. The number of determinations are reported without confidence intervals because they are not a sample of real determinations, but rather a graph.

4. Scope, trust assumptions, and operational predicates

4.1 Scope and threat model

An IAM role is suspected to be abusing the temporary credentials issued by its role in one of the organization's AWS accounts. Read only investigation can be independent in a scope of work that has been approved. The consequential operation is a deny policy that is a fixed-cutoff deny for an ordinary IAM role that has been pre-cutoff. The session-management procedures documented for service-linked roles or IAM Identity Center reserved roles are not included in OP-1, but are different [16]. The following operations are not admitted by this contract: Blocking future role assumption, deleting identities, modifying trust policies, changing the broker's privileges.

A compromised low-privilege role session can be used by an adversary, and retrieved notes and metadata can be affected by an adversary. The investigation agent can then suggest false targets, exclude contradictory evidence, or call for actions that are prohibited. In addition, the model covers benign withdrawal, resource-context invalidation, worker interruption, provider delay and unknown execution outcome.

The policy service, dispatcher, approval channel and ledger are trusted. At the boundary of the executable model, there are some assumptions such as: Evidence authentication, canonical identity resolution, clock correctness, and caller confinement. The Boolean fixture fields that are marked authenticated are the assumptions that are made; they are not signatures nor do they guarantee source integrity. If the assumed boundary is compromised by a malicious broker, administrator or direct theft of credentials from the dispatcher, the assumed boundary is invalid.

4.2 Requirement-to-mechanism mapping

In Table 2 the twelve requirements are kept and their state and actual implementation is indicated. “Refinement” refers to a design decision that has been made in the course of this study, but is not necessarily a new design in history.

Table 2. Requirement traceability and assessment boundary.

Requirement	Origin/status	Mechanism and assessment
R1: Traceable evidence	Adaptation of evidence-oriented investigation	ER-1, root identifiers, contradiction fields; 12 evidence fixtures
R2: Defined detector outputs	ML interface requirement [2,17]	Model/version/score fields; no detector-quality test
R3: Independent checking	Adopted enforcement [5,6,13]	Separate decision function; exhaustive permit-condition enumeration
R4: Evidence is not authority	Adopted agent-security principle [4,8]	Privileged facts distinct from note content; modeled prohibited-request cases
R5: Action-bound approval	Adaptation of contextual access [10,11]	Payload fingerprint and explicit withdrawal; approval fixtures and invalidation transitions
R6: Current-context validation	Adopted ongoing control [11]	Recheck before issue; late-effect boundary is separately reported
R7: Impact constraints	Cloud-response adaptation [18]	CR-1, protected-role rule, dependency assessment; 12 context fixtures
R8: Scoped identity and access	Adopted least privilege [5,10]	Exact role binding and restricted operation schema; deployment confinement not tested
R9: Bounded continuation	Accounting refinement	Serialized intervention admission; per-operation verification and handover tokens in composed tests
R10: Verified effects and recovery	Lifecycle adaptation [8]	Explicit unknown/handover states; modeled success and definitive-failure reconciliation

Requirement	Origin/status	Mechanism and assessment
R11: Auditable decisions	Design obligation	Linked proposal, payload, approval and execution traces; storage durability remains untested
R12: Balanced assessment	Study design	Positive completion witness, seeded faults, and reported unresolved outcomes

4.3 Typed input contracts

A proposal has the following elements: case_id, requester_id, operation_id, operation_type, account_id, role_arn, role_id, role_name, cutoff_utc, proposal_version, and evidence_ids. Identifiers are non-empty strings; account identifiers are 12 digits; versions are positive integers; the cutoff is a UTC timestamp. The operation type is a constant and not a command supplied by the agent. The composed entry point will return an error if additional fields are provided, if the type is not recognized, if the account/name/ARN bindings are inconsistent, if the canonical timestamp is malformed, if the evidence identifier is duplicated, or if the context age is not finite. The runtime checks are not replaced by Python annotations. The broker uses a template to build the policy, instead of taking arbitrary policy JSON from the agent.

The canonical payload fingerprint includes all the fields which are relevant to authority, such as the target, cutoff, case and proposal version. Canonicalization is not a claim of semantic equivalence of policy documents, but a limited JSON representation, sorted key and fixed serialization. The fingerprint is generated for the validated proposal and the exact generated request (policy name and body). This hash should not be used for caller identity and integrity authentication. A changed cutoff is a new proposal, NOT a redo.

The evidence interface is used to describe sources and source roots. The available fields for the operation, request context, and time vary by CloudTrail event; these can be preserved in a CloudTrail event using normalization [19]. Owner assessments that are independent are of their own origin and scope. The second alert is not a second independent observation, it is a reflection of the first alert. Record authenticity is a condition of admission, correctness of an admitted human statement is another uncertainty.

4.4 Evidence rule ER-1

The selected intervention is a deliberately conservative rule (ER-1). If the event or owner assessment is not authenticated, then the result is not known. If the target is not the same as the authenticated target, then false is returned. When there is an unresolved contradiction, then unknown is produced. If a predicate is an authenticated owner assessment and the predicate is true, and the predicate's corresponding change explains the predicate's operations, then the containment predicate is false. If the owner assessment has determined that the activity is unauthorized, the change has been

affirmatively determined to be unrelated, and the event and assessment have distinct nonempty derivation roots, then the result is true. All other combinations are not known.

Assessed unrelated is not a negative statement, it is a statement. Never does “no change record found” mean “corroboration.” Non-duplication is not at the level of statistical independence or truthful testimony, but at the level of the recorded lineage. The owner judgment should relate to the specific scope of the event and target. The responsibilities of deployment are authentication and delegation for that judgment.

This rule restricts the autonomy claim: compromise can be automatically established based on an already provided expert assessment, but it is not automatically established by the framework. In later work, a different documented policy may be used to replace ER-1 and the test results apply to that policy only.

4.5 Context rule CR-1 and approval rule AR-1

The first step in CR-1 is to take an authenticated resource snapshot. If a protected role or assessed critical dependency exists, then the intervention is false. If not, the role should be marked as test/production, dependencies should be affirmatively evaluated clear for this operation, expected context version should match and snapshot age should be within the range of 0 to 300 seconds. If information is missing, future-dated, stale or not resolved, you get unknown. The 300-second bound is not an empirically suggested cloud bound, but rather a reproducible fixture choice.

AR-1 makes denied or withdrawn approval false. It is possible to use explicit standing preauthorization in a designated test role. A production proposal must be accompanied by an authenticated approved record as well as the conditions of the approval. Approval status is unknown in the event of missing, pending, expired, unauthenticated or mismatched approval. Approval to execute is different from owner assessment as done by ER-1, as the two certify different things, even if the same person gives both assessments as per organizational policy.

The composed evaluation takes the explicit approval route to both test and production fixtures. The composed controller doesn't use the optional standing-preauthorization rule, but it is checked by the standalone AR-1 fixture group. Preauthorization override is not allowed for denied and withdrawn states.

The context predicate does not mean that all hidden dependencies are known, it just says whether the stated checks are true or not. Incomplete real-world information in the registry can lead to a sub-optimal response. That limitation in semantics is not converted to an extra confidence measure that is tested, but is reported.

5. Three-layer architecture and concrete cloud-operation contract

5.1 ML detection, agentic investigation, and controlled response

The framework is organized as three layers. Layer 1, ML-based detection, receives normalized telemetry and produces versioned findings that preserve detector identity, observation window, score meaning, feature provenance, and evidence references. Layer 2, agentic AI investigation,

operates through a bounded tool catalog to request permitted evidence, compare competing explanations, and produce a typed response proposal. Layer 3, controlled autonomous response, independently evaluates the proposal, evidence, resource context, approval, capacity, dispatch state, and observed effect. Figure 1 maps the seven logical responsibilities across these layers and marks the executable abstraction evaluated in this article. Detection and agent planning are upstream interfaces rather than executed experimental components. The executor owns any future cloud credentials; the agent is not a credential-bearing administrator.

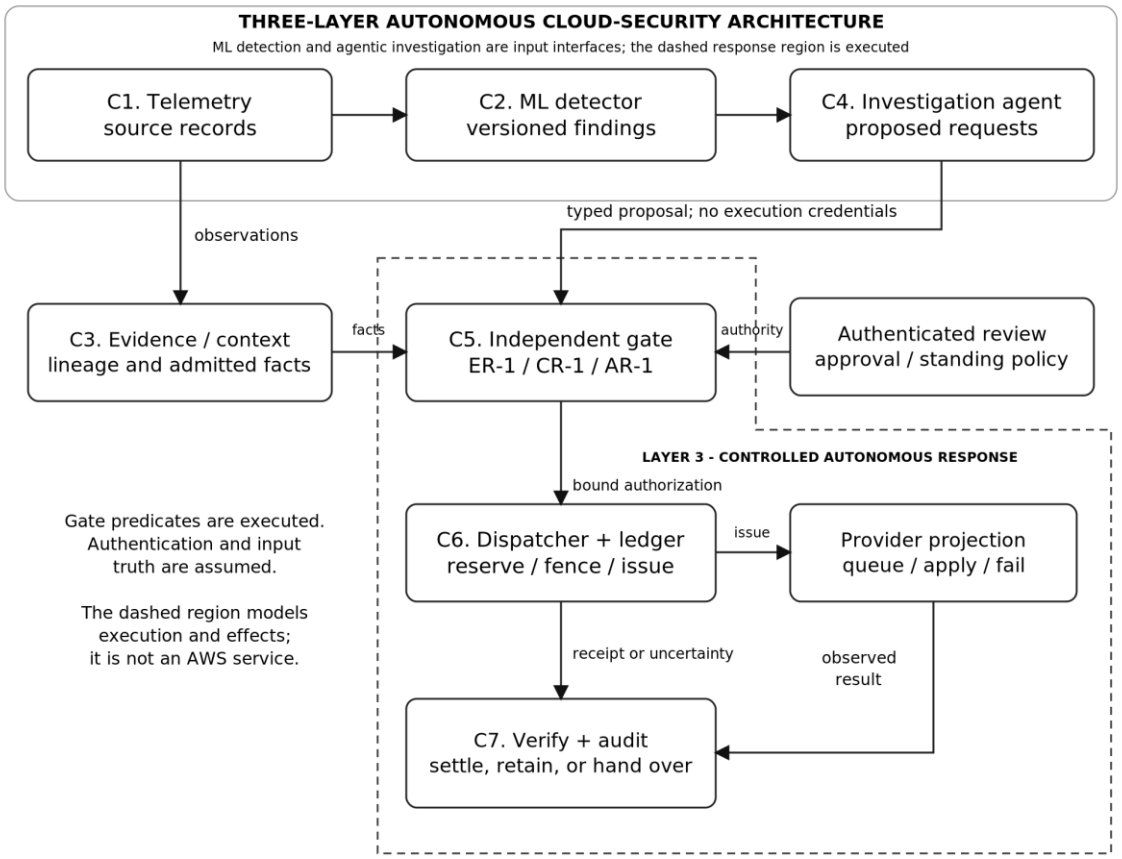


Figure 1. Three-layer architecture: ML detection and agentic investigation feed a separately authorized autonomous-response layer; the dashed region marks the evaluated abstraction.

Unlike the framework, at the ML-to-agent boundary, it does not convert a detector output to a bare alert label. It preserves the detector/model version, observation window, affected role, score definition, feature provenance, supporting events, and unresolved problems with the input quality. One of the families of detectors is called Isolation Forest, and the original method was mentioned instead of using an API page as the algorithm's research source [17]. There is no detector selected, trained, calibrated or compared in the present evaluation. An anomaly score might be given as a priority for investigation but does not satisfy ER-1, CR-1 or AR-1 and thus may not generate execution permission.

At the agent-to-control boundary, the investigation agent is given the finding and access-filtered context, and he chooses which queries are permitted and generates a structured proposal, which

includes the target, the operation, the parameters, the evidence references, the uncertainty that remains, and the proposed verification. The agent can tailor its investigation sequence, but not install tools, expand credentials, or authorise his/her own response. It is important to note that the composed control tests are deliberately designed to fix the candidate proposals, and the study is designed to assess whether the agent-generated proposals would be bounded correctly, and not whether the agent generates good proposals.

5.2 Operation OP-1: deny pre-cutoff role sessions

OP-1 applies a deny policy that is generated by a broker to one specific IAM role. GetRole returns the ARN, unique identifier, name and trust-policy information for the role [20]. PutRolePolicy adds or updates an inline permissions policy, using a role name, policy name and policy document [21]. A preflight associates the account, ARN, unique role identifier and name to the approved target. The adapter does not make any assumptions about identity based on the display name.

The study names a policy based on an immutable operation identifier, and assigns it to each operation. This design does not simply replace an existing incident's policy. An error if there is a collision with another name and different content. A production adapter is required to verify existing named policies, while the local harness verifies any logical identifiers and payloads, but not a cloud inventory. For per-operation naming, there's also a quota and clean up policy. AWS does not allow more than 10,240 characters (excluding whitespace) for a role's total size of all inline policies [22]. If the normalized ASCII size of the new body, if added to the body that has already been admitted into the contract, would exceed this limit, the composed contract will reject a new body. Deletion is still an operation that is approved separately and not a shortcut for automatic deletion. The following deny body is generated by the dispatcher, and the cutoff is not changed until it is approved. This is an example specification artifact that was not used in an AWS account.

```
{
 "Version": "2012-10-17",
 "Statement": [{
  "Sid": "DenyPreCutoffSessions",
  "Effect": "Deny",
  "Action": "*",
  "Resource": "*",
  "Condition": {
   "DateLessThan": {
    "aws:TokenIssueTime": "2026-09-06T10:00:00Z"
   }
  }
```

```
    }]
  }
```

The wildcard action and resource are in a deny statement on the target role, and are not granted to the broker. Explicit deny takes precedence in the evaluation of policies in AWS documents [23]. Note that the actual broker permissions need to be set to limit the target role and registered API separately. Calling PutRolePolicy does not guarantee the integrity of the fixed body in the broker, nor does it guarantee that no arbitrary requests are included in the template, it is still a trustworthy task for the broker to guarantee.

The logical effect is intended to be more limited than “disable the account.” AWS role-session revocation is based on a time stamp conditioned deny, and does not apply to sessions assumed after the revocation time stamp [16]. DateLessThan is used to compare a date to another date, and it is strictly less than the other date [24]. The generated policy is read by the composed evaluator, and it reads the comparator, key and cutoff from the generated policy itself and evaluates pre-cutoff, equal-cutoff, post-cutoff and delayed-visibility cases. A separate validator will be needed, and must be written with the same deny template and approved cutoff. The evaluator is not a full IAM engine, but is a subset of this limited grammar. It is not a service or resource-policy model and neither are the legacy timestamp-only helper models.

5.3 Parameters, preconditions, and authority

Table 3. OP-1 contract and concrete verification obligations.

Contract element	Specification
Target	One approved account, role ARN, RoleId, and RoleName; no wildcard target
Caller	Dedicated executor identity with the required role-scoped permissions; no agent-held credentials
Mutation	PutRolePolicy with a fixed, broker-generated deny template and immutable UTC cutoff
Preconditions	ER-1 and CR-1 true; valid AR-1; policy-name availability; capacity reservation; dispatcher and audit readiness
Human route	Production requires action-specific approval; hard prohibitions cannot be overridden within the case
Retry policy	One application-layer request attempt; no SDK/proxy retry and no reissue after ambiguity
Configuration check	Read the named policy, normalize the returned JSON, and compare target and expected body

Contract element	Specification
Effect check	In future isolated testing, use a permitted canary session issued before cutoff and a separately bounded probe
Non-goals	Blocking future assumption, account-wide containment, erasing credentials, or proving absence of compromise
Recovery	Separately authorized removal or replacement; never automatic deletion solely because approval was withdrawn

GetRolePolicy returns the named inline policy; the API representation may need to be URL decoded for comparison [25]. A read-back is to confirm the returned configuration, not global enforcement. IAM propagation is eventually consistent and can include cached observations [26]. Thus, in an implementation, bounded observations must be used and a record must be made of the specific effect that was tested. An old session that was never allowed to run the probe cannot be used to attribute an access to OP-1 as a positive control for a future canary experiment; there should be a pre-intervention positive control.

The registered action is not specific to an identified attacker session, but rather it is broad, and may impact all covered pre-cutoff sessions. Legitimate sessions must be therefore also included in an owner assessment and dependency review. A new role session can be made after the cutoff and leave a separate access path. An intervention that prevents that route is not a part of the separately assessed intervention.

The retries need to be under the adapter in the issue boundary. The total_max_attests=1 parameter is the number of attempts that are made, including the first attempt, and that SDK retries are disabled [27]. The potential adapter contract must then include both setting and prohibit individual proxy / application retries. The composed harness checks does not create an SDK nor does it certify transport conformance. Retransmission of a packet in the same request is not considered to be a new application-layer mutation attempt.

5.4 Identity replacement, other writers, and rollback

The PutRolePolicy request documented does not have a role-GUID compare-and-swap or client idempotency parameter [21]. The broker can check the RoleId before and after a request, but the checks are not a condition at the time of the provider mutation of an immutable identity. If the operating conditions or the deployment risk is not resolved, the following exclusion must be applied: Role deletion/recreation and conflicting administrative writers must be excluded. That's a human-approval limitation of the interface and it cannot be fixed.

A new response is based on a new policy name and has a fixed cut-off date. Even after an ambiguity, a duplicate write is not automatically retried, even if it appears to be repeatable: a duplicate write may have been delayed, and restored the restriction after the operator removed it. The evaluated

design opts for accountable handover over inventing exactly-once behavior. This compromises availability for a well-defined automatic route.

RemoveRolePolicy deletes an inline policy [28] but a deny may be removed and re-grant access to credentials which are still valid. Therefore, rollback is not a mechanically triggered reversal of the decision based on an unsuccessful verification, but another decision which has its own evidence and approval. The contract will include retention of the previous and observed policy states, and a clear evaluation of the irrevocable effects. These are design requirements and not outcomes of the model tests.

6. Authorization and recoverable dispatch model

6.1 Decision semantics

Make a proposal a = (case, requester, operation, target, parameters, evidence, version), where each of the authority fields is bound by the fingerprint of a proposal. Assign T, F or ? to each condition in G={V,P,E,C,A,B,O} V is the validity of the request; P is the scope of policy; E is ER-1; C is CR-1; A is the authorization path; B is capacity and sequence compliance; and O is operational readiness. The accounting variable below is not represented by the unknown symbol ?. The callable gate will check the exact set of keys before evaluation and will ensure that each value is a real Truth enumeration member. A controlled validation error is raised if the string to be validated is null, or is a raw string, or is a Boolean, or is a number, or is a list, or is a mapping.If the string to be validated is null, or is a raw string, or is a Boolean, or is a number, or is a list, or is a mapping, a controlled validation error is raised and no malformed state reaches the permit branch.

Eligibility is a definition, not a novelty claim:

Eligible(a,c) = 1 iff every g in {V,P,E,C,A,B,O} equals T. (1)

The local conjunction is false-dominant; that is, if the local conjunction is F, then it is F; otherwise, if the local conjunction is ?, then it is ?; otherwise T. An unsuccessful V, P, E, or C is rejected in the outcome procedure first. It then transfers the approval that has been denied or withdrawn, identity or scope of the application and unavailable controls, and insufficient applicable capacity. An evidence request is only generated for an unresolved E or C if there is a still pending resolving path. If the other conditions are true, then there is an approval request if any of the following is false: All T means that you are eligible. Permission is never given for an unclassified condition.

This procedure is intentional to separate the true/false result of a predicate from a workflow result. A predicate that is known to be valid but changes for a known reason can cause a containment predicate to be false without closing the incident. Handover does not mean that the investigation was malicious, it can be an unknown investigation. Approval is a condition that must be met, it is not a substitute for all other conditions as stated in the policy.

6.2 Shared accounting and operation identity

Let L be a fixed nonnegative allowance, D the consumed amount, R the reserved amount, and w(a) a nonnegative operation charge. Admission requires:

D + R + w(a) <= L. (2)

The bound is about the set-up of intervention units, rather than harm measured. All of the above are performed in a single transaction that is serialized across all operations that share the allowance. If a duplicate logical identifier with the same owner and payload is found, it returns the same state as before. An alternate payload is rejected. If there are operations that might conflict on the same role, you need to use a per-target exclusion rule. If an operation is not sent, it will cancel its reservation. A confirmed effect will move the charge from reserved to consumed. An operation that is not certain or handed over keeps its reservation, as it might still have an effect. One verification token, and one handover token are also reserved per operation in a separate follow-up allowance. Those tokens can't be used by the investigation method. The verification uses up its token and handover keeps the intervention reservation. It is not reset during the unresolved operations. A multi-scope budget or rolling window budget needs extra semantics, the present model does not demonstrate them. Per-resource ledgers are not necessarily coordinated to impose a shared account-level bound.

6.3 Lifecycle and ownership

The operation lifecycle is shown in Figure 2. PREPARED indicates the request and the reservation are there but no dispatch intent is committed. In the event of a failed preparation owner, a recovery coordinator will take over with a monotonically increasing epoch. The dispatcher is responsible for discarding old epochs. Lease timeout is not enough of a fence as an old worker can come back. The composed dispatcher validates the provided epoch in both intent and issue approaches. A 2 worker recovery is done by the new owner and rejection by the old owner on both boundaries. That ownership enforcement is an assumption in the smaller exhaustive transition model, passing composed traces are not a refinement proof connecting the two models.

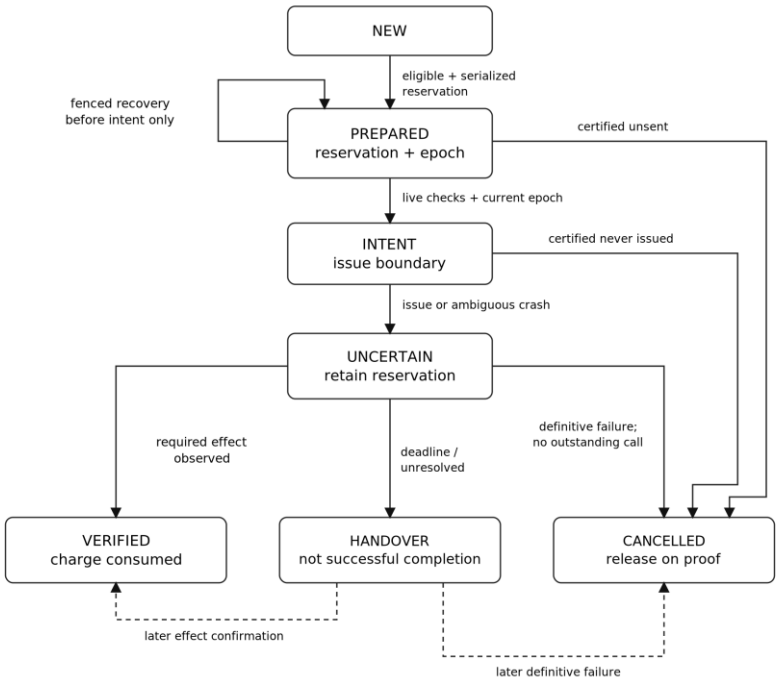


Figure 2. Preparation is recoverable; ambiguous intent is reconciled, not automatically resent.

The request is at the edge where it could be issued externally, as per INTENT. If the crash occurs around that time, it may result in a situation that is indistinguishable between “not sent” and “sent but unacknowledged”. This request is UNCERTAIN if the dispatcher cannot certify that he/she did not make the request. A returned PREPARED record cannot be made into an unrestricted retry. A named coordinator comes into play only for pre-intent work; post-intent uncertainty is introduced in reconciliation.

A provider request can be queued, applied or definitively failed. The broker's state is not related to the provider's transition. If the modeled effect is observed to be positive, then it is VERIFIED. A definitive failure, without any outstanding duplicate permits cancellation. If there is a reconciliation period, it results in HANDOVER but the reservation remains. HANDOVER does not end the autonomous attempts, but the incident itself and the potential of a late provider effect. A later authorized reconciliation may resolve the issue.

Most of the modeled dispatchers issue at most once after INTENT. This is not a protocol that guarantees exactly-once delivery of an AWS API, but rather a protocol choice that is restrictive. It avoids the unsafe retransmission in the automatic path, and makes the availability cost explicit.

6.4 Dispatch procedure

Algorithm 1. Operation-specific preparation and dispatch protocol.

1. Validate requester, case, typed target, operation and fixed payload.
2. Resolve admitted evidence, policy, context and approval.
3. Apply the ordered gate; return non-permit outcomes with reasons.
4. In one ledger transaction: reject changed duplicates; check capacity;

   reserve the charge; store PREPARED, payload and owner epoch.
5. A coordinator may reclaim PREPARED using a newer fenced epoch.
6. The dispatcher checks owner epoch, live approval and relevant context.
7. Commit INTENT for the exact request; prevent competing issuance.
8. Before an unambiguously unsent issue, reject observed invalidation.
9. Issue once through the registered adapter and record the result.
10. On ambiguous intent or receipt, enter UNCERTAIN; do not reissue.
11. Observe the registered effect or definitive failure; otherwise hand over.
12. Settle verified effects or retain uncertain reservations and audit state.

No step can atomically commit an external provider effect inside a local ledger. The full model therefore asserts valid authority at issue, not at every later instant. An approval withdrawn after issue is a material event to record, but does not prove that an already queued request has been canceled.

6.5 Safety properties and conditional progress

There are five invariants which are checked by the finite exploration. H1 is only issued if D+R<=L; H2 is not issued if the modeled current approval or context are invalid. Modeling the external effect is not allowed for H3 and VERIFIED. H4 does not allow canceling or releasing as long as there's an effect queued or applied. The number of H5 bounds issued is limited to one per logical operation. H2 is for approval/context revalidation on the basis of initial eligibility; the evidence and identity predicates are used separately by the gate and fixture tests.

Suppose that the initial accounting is valid, all charges are nonnegative, the admissions are serialized, and the settlement rules given above apply for H1. The bound is preserved by induction over those transitions, and an increase in R is only possible by the transition to R via Equation (2), while a decrease in R is possible by the transition to D via verification, and the transition to D via handover preserves R. This is not a new resource-control theorem, it's an ordinary conservation argument. The charge might be underestimated and bring about significant damage without breaching the H1.

Progress is conditional. With a stable coordinator/dispatcher, a stable eligible PREPARED operation has a finite path to issue. Eventual observable provider resolution: UNCERTAIN is verifiable or can be definitely cancelled. If not, a working deadline/handover process can prevent autonomous efforts from taking off while still ensuring accountability. The artifact does not verify fairness in arbitrary scheduling, repeated failures or unbounded arrivals.

7. Executed model-level evaluation

7.1 Implementation and finite bounds

All results are obtained with the distributed scripts using Python 3.13.5 on Linux. Artifacts are independent of any network or cloud SDK. The predicate tests are used to check whether the facts are structured in a way that is privileged; the composed interface is used to validate the structure of the facts, but not to verify the actual facts. The provider model does not support all IAM semantics, it is a fixed-cutoff deny and asynchronous application.

The state explorer has two operations with different targets, and two unit charges as well as fixed allowances of one and two. Approval and context may expire, once, and are not renewed. At most one preparation-epoch takeover and one external issue are allowed per operation. Provider completion can be after local handover or invalidation. Within these limits there is no limit on search depth. What's not in the graph is: self-loop stuttering, wall-clock time, unbounded requests, same-target policy conflicts, and interacting multi-scope ledgers.

The full model is compared with three fault seeded versions with the same initial inputs and transition opportunities. Snapshot takes out the live issue check. Split-admission is a technique that splits up an earlier capacity check from a reservation. Release-unknown allows an unknown reservation to be thrown away. These variants detect mechanism-level faults and do not predict the behavior of any published agent system.

7.2 Predicate and decision results

Thirty-two fixtures were created, each providing the specified values: 12 evidence cases, 12 context cases and eight approval cases. The cases feature a supported response, a legitimate change, incomplete change matching, common evidence roots, conflicting records, untrusted assertions, target substitution, stale context, negative age, protected roles, critical dependencies, mismatched approvals, withdrawal and expiry.

There are 2187 combinations of the seven three-valued conditions, which were all enumerated. Only one condition assignment can be used: all seven T. The decision function performed the same action as the all-true permit expression on each assignment. Other claims include withdrawn and denied approval and unavailable resolving paths. This is not a statement of the incident truth of the predicates, but of conformance to the stated gate.

All 66 faulty gate-input probes were turned down. There are nine invalid value types at seven condition positions and three invalid key sets. There are 15 previous local sanity checks that pass as well, and they are included in the supplement, but are not considered to be integrated lifecycle evidence.

The composed harness passes 29 schedules. These are: full approved request generated by policy, reservation, issue, application, and verification, stale, replacement workers with the actual dispatcher, target substitution, cutoff substitution, case substitution, requester substitution, approval invalidation, context invalidation, unsupported role classes, exhausted policy capacity, exhausted follow-up capacity, and verification after investigation capacity is exhausted. The full proposal, body, binding, phase history, and issue count and accounting disposition are stored in each case.

The real generated policy is used in four semantic boundary checks. Prior to reservation, all ten structural policy mutations are rejected. Two additional tests modify the source of the generator in disposable directories: DateLessThan becomes DateGreaterThan and the approved cutoff is changed. Both cause the policy-contract check to fail. This resolves the observed test gap where a helper with a timestamp could pass, but a generated policy could have passed a different condition. The intentionally introduced mutants test these failure modes, but not the probability of general fault detection.

7.3 Reachable-state results

Table 4. Exhaustive reachable graphs for the stated finite configurations.

Variant	Limit L	Reachable states	Transitions	Checked-invariant violation found
Full	1	4,140	12,716	None within the model
Full	2	15,376	64,232	None within the model

Variant	Limit L	Reachable states	Transitions	Checked-invariant violation found
Snapshot	1	7,680	23,632	H2: invalid issue
Snapshot	2	29,584	126,936	H2: invalid issue
Split-admission	1	16,384	68,140	H1: over-allocation
Split-admission	2	16,384	68,352	None; two unit operations fit
Release-unknown	1	10,668	40,180	H4: unaccounted outstanding effect
Release-unknown	2	21,904	99,752	H4: unaccounted outstanding effect

These two complete configurations have a total of 76,948 transitions and 19,516 states. These are sums of separate graphs, not that there are that many distinct real-life situations. None of H1–H5 was violated in those graphs. The check does not show exactly-once cloud behavior, and is enforced by the selected no-reissue transition topology.

The lack of a split admission failure at allowance two is informative and is expected. No two unit operations can have a value greater than that bound. This non-failing variant does not make it appear that a removed safeguard is required to fail for all input sizes.

7.4 Reproducible counterexamples

Withdraws approval, records intent, and issues. It is not an AWS IAM permission error, it is violated property at issue. The full model eliminates that transition when invalidation is seen.

The split-admission trace allows both operations the chance to see what capacity is left before they reserve it. Both then reserve, more than one allowance. This particular interleaving is avoided by serializing each reservation with its admission check. This result is not a statement of a database implementation's isolation level.

The release-unknown trace allows for the admission, recording of intent, recording of issues, and the release of the reservation on uncertainty when a provider request is queued. An after-effect would be unaccounted. This is an accounting error that can be avoided by retaining capacity, but can result in a permanent loss of availability if a reliable observation is not possible.

Table 5. Minimal counterexample schedules at allowance one.

Condition examined	Generated schedule	Interpretation
Snapshot authorization	Admit -> withdraw -> intent -> issue	A previous permit cannot establish current issue authority
Split capacity check	Admit 0 -> admit 1 -> reserve 0 -> reserve 1	Independent observations do not conserve a shared allowance
Early uncertainty release	Admit -> intent -> issue -> release	A timeout is not proof that an effect cannot occur
Full-model late effect	Admit -> intent -> issue -> withdraw -> provider apply	Valid issue does not guarantee authority remains valid at effect

The last row is not a fault that is seeded into the full control model. The explorer finds it in both full configurations. H2 remains valid, since it was approved at issue time. The counter example is used to show that an overbroad effect-time interpretation of the broker guarantee is false. Approval does not provide a remote cancellation/conditional effect mechanism.

7.5 Relation to the six scenario families

S1 corroborated misuse has a positive permit and completion path. S2: Legitimate administration denies the specific unsupported containment basis, but does not automatically close the case. S3 - conflicting evidence - retains unknown and does not count duplicate roots as corroboration. For malicious content (S4), prohibited requests and untrusted structured facts are represented, and there is no language-model prompt-injection success rate measured. Invalidation transitions are directly represented in S5, where they are either an approval or context change. Delayed application, definitive failure, intent uncertainty and accountable handover are all examples of S6, ambiguous execution.

These mappings are used to make the original narrative scenarios reproducible in specific components, but do not provide an end-to-end agent investigation. The artifact doesn't produce plans, natural-language reports or get owner judgments. It tests the action of the control logic when the appropriate structured input or request is provided.

7.6 Interpretation of the completed evidence

The findings were limited to three. The gate has been installed as part of its published permit condition. The complete finite transition model maintains the above-mentioned invariants within its bounds, whereas some weakened variants reveal the targeted faults. The same model shows a restriction on guarantees over asynchronous external effects.

None of the results provide any benefit in detection accuracy, analyst workload reduction, resistance to any prompt injection, production reliability or superiority over TraceGrant, Progent, Agentra or a commercial platform. These methods and counts are not an alternative for those unique evaluations, but are computational observations.

7.7 Base-paper results and comparability

Selected results reported in the base paper are reported in Table 6 and not experiments rerun here. It is mainly composed of 54 tasks and 270 agent executions. The CommitGuard row is an independent comparison of the enforcement of 18 tasks. These denominators should not be used with our finite-state counts or twelve local schedules.

Table 6. Base-paper reported results; source [9], Tables 8 and 15.

Source experiment	Visible completion	Authorized completion	Invalid effects	Other outcome
Primary unguarded matrix	262/270	55/270	207/270	8/270 safe non-completions
CommitGuard enforcement: 18 clean + 18 perturbed	18/36	18/36	0/36	18/36 guarded aborts

Our results are for a different experimental object: a fixed contract and controlled schedules without plans generated by agents. There is no gain in accuracy, speedup or percentage improvement over the base paper calculated. It is not the success or authorization that is reported that drives our reporting categories, but it is the reported success and authorization that is what makes it motivating. The original monitor was not reimplemented and an original benchmark task was not run.

7.8 Same-input comparison of authorization boundaries

The 12 controlled schedules are applied to 3 local controllers in a separate controlled comparison. They all share the same concrete proposal, evidence, policy body, quotas, ownership checks, provider projection, and phase resources. The only difference is that of the timing of authorization enforcement. The schedules contain three authority-preserving controls (ordinary completion, benign delay, prepared-worker recovery), four invalidations before issue, three invalidations after issue, one definitive provider failure and one ambiguous no-send case.

Snapshot validates at admission, and excludes the later authority verdict at intent and issue. It is not the unguarded agent of the base paper, but is a deliberately weak local reference, not a published product. The proposed composed controller is issue-time: It rechecks the contract before it's issued, but it can't recall a queued provider request. Atomic-reference adds a simulator-only validity check, along with provider application. It is not a faithful CommitGuard reproduction nor a capability of OP-1, it's a paper-guided idealization with a stronger primitive.

The same event order and admitted inputs are given to each controller for each schedule. If the schedule has previously invalidated the authorization or context, then the applied effect is considered invalid according to the independent outcome rule. The reference labels are not determined by the verdicts of the controller. The external state was applied at the endpoint (endpoint completion) and valid authority was also present at that time (authorized completion). Safe cancellation, definitive failure and unresolved handover are distinguished in the detailed traces and are considered non-completion.

Table 7. Newly executed local comparison: twelve identical schedules per controller.

Local controller	Invalid issues	Applied effects	Authorized completions	Invalid effects	Non-completions
Admission snapshot	4/12	10/12	3/12	7/12	2/12
Proposed issue-time	0/12	6/12	3/12	3/12	6/12
Ideal atomic-reference	0/12	3/12	3/12	0/12	9/12

The proposed controller invalidates all four pre-issue invalidation schedules that it is missing from the snapshot reference, but keeps all three authority-preserving completions. It doesn't stop the three effects for which the conditions are invalid after issue. The atomic prevents those three effects since its provider-side primitive changes what is enforced. One controller will have an unresolved handover, that will not be reported as a proven safe termination.

The comparison does not make it a fact that the proposed system is superior to the base paper. Instead, it generates the predicted capability boundary in a specific specialization and measures its performance on the same locally generated inputs. The atomic reference has a stronger stance on effect-time integrity, and the proposed controller's defenseable claim is an explicit issue-time contract and recovery/accounting semantics for the weaker OP-1 interface. These are not deployment rates, nor are they samples of incidents, nor do they represent statistically estimated improvements.

7.9 Coverage and negative evidence

7.9 Coverage and negative evidence

The passing results set different properties at different layers. Valid-domain decision conformance is checked by gate enumeration, malformed probes are checked by one public input boundary, generator mutations are checked by policy-contract sensitivity, composed traces are checked by selected connections and graph exhaustion checks the smaller transition abstraction.

The source-mutation probes and the late-effect schedules are particularly useful as they may contradict a too general success statement. The former find that the implementation has changed; the latter show a residual limitation, not exclusion, in order to get a better score. There are two clean

runs and file-level hashes, but no claim of independent authorship or formal verification to ensure reproducibility.

8. Deployment obligations and next-stage evaluation

8.1 What a production implementation must still establish

A production broker is required to enhance the modeled topology: agent processes must not have direct mutation credentials, admission and reservation must be serialized over the actual allowance scope, epoch checks must be performed at the sole dispatch path and durable storage must withstand the relevant failures. The present in-memory immutable state transitions are not testing the durability of the database, the confinement of the network, the isolation of credentials, or the multithreaded code.

Other provider-backed validations include role identity binding, event integrity, authenticated expert assessments, policy retention, and read-back or canary probes. Any administrative changes that are external to the broker should be limited by the operating model or explicitly added to a new threat model. Read-only investigation is required to prevent the disclosure of sensitive information and output restrictions are required as part of the process.

Ambiguous intent recovery and prepared state recovery should not be combined. An operation that has been verified can be reassigned to a new epoch. An ambiguous one cannot be resolved just because of the expiration of a lease. If there is doubt, human handover keeps the reservation and a separate incident-governance process determines the next course of action, either to expand capacity or to select another course. It is not acceptable to make such a ruling without explicit mention of what was previously approved.

8.2 Availability, fairness, and resource windows

The fail-closed writes can cause a delay in the response to the urgent calls. Having all unresolved reservations in place does not cause an accounting failure, but does create a denial-of-service opportunity if there are a lot of these reservations. The first model is explicit about this: it reports handover and doesn't release capacity on timeout.

A later service requires queuing, backpressure and a separate authorized emergency process. Also requires an explicit rule for allowance changes and accounting-window rollover. The fixed-bound proof does not warrant the reset of consumed units as previous requests may still be in effect. If there are more than one account, case and resource allowances to be reserved, they must be reserved as a group, or the protocol must indicate which invariant it gives up. They are needed extensions and not features that are assumed to be provided by the existing scalar model.

The composed harness verifies that there are resources in the follow-up that are not required for investigation, including a successful verification once investigation is complete. These unit cost schedules do not give any idea of their useful real world sizes. Nor is there a finite successful recovery witness that sets fairness between low-priority and urgent incidents.

8.3 Separate enforcement from agent-quality experiments

The next enforcement experiment should replay identical proposals, evidence updates, approval schedules, and failure events through competing control implementations. Reference policy and acceptable outcomes must be specified separately from the implementation being evaluated. Outcomes should distinguish contract-invalid issue, cloud-permission violations, late effects, false verification, unnecessary blocking, and unresolved operations. This prevents a stricter local rule from being credited automatically with preventing an IAM authorization violation.

A separate agent experiment should compare fixed playbooks with dynamic investigation while retaining the same control layer, tools, detector findings, and resource limits. It should measure newly discovered relevant evidence, unsupported claims, acceptable responses, justified escalation, verification, and resource use. SIR-Bench provides a precedent for measuring investigation depth rather than only final triage labels [29]; its benchmark does not validate this containment contract.

Model-generated plans should not define their own ground truth. Reference judgments should be independently reviewed, and variants from the same incident pattern should remain in one data partition. Repeated model runs are not independent incident patterns. Real analyst-effort claims require an appropriately authorized human study. None of these additional experiments is reported as completed here.

9. Discussion and threats to validity

9.1 Meaning of the contribution

The article's title refers to the complete three-layered solution: detection via ML, investigation via agentic AI, and autonomous cloud response. The computational evaluation is intentionally narrower in that it tests the control and execution layer, and the typed boundary where an agent proposal is passed into that layer. No new method of learning is claimed and no inference of agent quality improvement is made. The ML finding contract, the bounded agent proposal interface, ER-1, CR-1, AR-1 and OP-1 answer RQ1. The lifecycle, the graph transitions between ledgers, the graph exploration that is performed, and the explicit counterexamples address RQ2.RQ2 is answered by the lifecycle, the transitions between ledgers, the graph exploration performed, and the explicit counterexamples. RQ3 is tackled by splitting up model conformance, dispatch authority, eventual effects, semantic judgment and future detector/agent evaluation.

The most recent frameworks that already link evidence and authority are quite close. The smaller purpose of this article is to provide a concrete interpretation of the cloud-role, executable failure schedules, and exact bounds on a popularly desired guarantee. Effect-time authority is already set forth in the base paper. What we need, then, is not a rediscovery of that principle, but a comparative evidence that is a special capability analysis. The finite results do not create a complete novelty or a level of production. They simplify the proposed design to be easily falsified and extended.

9.2 Internal validity

The same development process resulted in the specification, code, fixtures and expected results, which comprise the locally written comparative schedules. A common misunderstanding thus can

withstand all tests. The artifact was implemented and the results analyzed, but there was no third-party validation or code review. Selected checks only demonstrate that selected faults can be detected using model mutation; it does not prove that the fault set is representative or complete.

Predicate tests are based on facts admitted, not cryptographic records. The authenticated flags, context version equality and owner dispositions might be incorrect in deployment. In particular, the independent origin check carried out by ER-1 is not a truthful assessment. The three-valued gate can always give permission to respond inappropriately if its trusted inputs are always misled.

9.3 Abstraction and external validity

The finite graphs limit operations, costs, invalidations, owner recovery and provider issues. They do not support arbitrary retries, policy restoration, repeated expiry, mixed operations, unbounded queues, concurrent same-role writes or multiple budget scopes. Exhaustion is only used with finite graphs. This Python representation is not a verified compilation of the prose specification and has not been verified using an independent model checker.

The provider projection now includes the actual restricted policy generated and separately represents visibility and asynchronous effect. Does not include complete IAM evaluation, service-specific exceptions, cached authorization, cross account resource policies, or actual role-lifecycle races. A modeled APPLIED state is more robust than the evidence that a real remote read might provide. The artifact is thus a proof of internal transition behaviour instead of AWS effect correctness.

Literature analysis is focused and contains preprints. References indicate the inspected versions, but they are not a conversion of the author's reported results to independently confirmed results. Further work might be done that further restricts the claim. The comparison should be updated as of the date of submission.

9.4 Residual semantic and operational risks

An attacker can incorporate malicious activity within the allowed tools, for example, a partial report. A corrective approval can be given on the basis of a misguided summary. An operationally justified action can be unsuccessful following dispatch. Better accounting doesn't solve those problems. The framework helps to keep them visible and helps to differentiate them from authorizations violations, but it does not promise to eradicate them. Conservatives' no reissue rule may prevent a valuable action from being completed when a car accident occurs. It's a clearcut compromise. A future retry design would require evidence from the provider and a new state model instead of an assertion of idempotence. Similarly, if OP-1 was expanded to block future sessions, the effects would be altered and a new contract and assessment would need to be done.

10. Conclusion

This article brings a three-layer design for autonomous security agents in cloud environments. Versioned findings and evidence references are provided by the ML-based detection component, bounded tools are used by the agentic AI to research findings and suggest actions for them, and an independent response layer determines if a suggestion can be authorized, issued, reconciled and verified. The third layer and the boundary between the agents and controls is the focus of the

executable study, which is implemented using a concrete IAM role-session intervention, keeping the distinction between intelligence generated by the model and authority to execute it.

The completed artifact satisfies 32 predicate fixtures, 2,187 valid-domain gate assignments, 66 malformed inputs, 29 composed schedules, 10 policy mutations, 2 generator-source mutations and 8 finite transition configurations. The separate 36-run comparison uses twelve schedules that are matched between the three local authorization mechanisms. All of the abstract configurations maintain five stated invariants in 19,516 reachable states and fault-seeded variants reveal stale authorization, concurrent over-allocation, and premature uncertainty release. Even with the full model, there is a possibility that a late-effect trace will still exist, which is why dispatch authority can't be promoted into an unconditional effect-time guarantee. On the matched schedules checking of issues before they are made means that 4 issues are invalid, but 3 issues are invalid after they are issued. Those effects are only prevented by a stronger atomic reference. The contribution is not just that machine learning or agentic AI is a better approach to cloud security, but that they can be used as an operational integration pattern in which their outputs can be used while maintaining explicit boundaries of authority, recovery, and uncertainty. The operation specific specification, associated test path and reproducible computational evaluation does not imply superiority to the base paper. No claims are made about an AWS deployment, about improving the performance of learning-models, about independent expert validation, or about production-safety. The next research step is to link a trained detector to a real investigation agent to the same control layer and to compare the dynamic investigation to a fixed playbook in the same control.

References

[1] Nelson, S. Rekhi, M. Souppaya, and K. Scarfone. Incident Response Recommendations and Considerations for Cybersecurity Risk Management: A CSF 2.0 Community Profile. NIST SP 800-61 Rev. 3, 2025. doi:10.6028/NIST.SP.800-61r3.

[2] Amazon Web Services. What is Amazon GuardDuty? Amazon GuardDuty User Guide. Accessed September 6, 2026. https://docs.aws.amazon.com/guardduty/latest/ug/what-is-guardduty.html

[3] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao. ReAct: Synergizing Reasoning and Acting in Language Models. ICLR, 2023. Author version arXiv:2210.03629v3. https://arxiv.org/abs/2210.03629v3

[4] E. Debenedetti, J. Zhang, M. Balunovic, L. Beurer-Kellner, M. Fischer, and F. Tramer. AgentDojo: A Dynamic Environment to Evaluate Prompt Injection Attacks and Defenses for LLM Agents. Advances in Neural Information Processing Systems 37, Datasets and Benchmarks Track, 2024. Author record arXiv:2406.13352. https://arxiv.org/abs/2406.13352

[5] OWASP Gen AI Security Project. LLM06:2025 Excessive Agency. Accessed September 6, 2026. https://genai.owasp.org/llmrisk/llm062025-excessive-agency/

[6] T. Shi, J. He, Z. Wang, H. Li, L. Wu, W. Guo, and D. Song. Progent: Securing AI Agents with Privilege Control. arXiv preprint 2504.11703v3, May 14, 2026; originally submitted 2025. https://arxiv.org/abs/2504.11703v3

[7] R. Patel, S. Mitra, M. Guida, S. Iannucci, S. Mittal, and S. Rahimi. Agentra: A Supervisable Multi-Agent Framework for Enterprise Intrusion Response. arXiv preprint 2606.18325v2, June 18, 2026. https://arxiv.org/abs/2606.18325v2

[8] Liao, J. Wang, Q. Song, J. Cao, J. Wang, and B. Deng. TraceGrant: A Contract-Governed Security Framework for the Task-Effect Lifecycle of Networked LLM Agents. arXiv preprint 2608.21126v1, August 21, 2026. https://arxiv.org/abs/2608.21126v1

I. Santos-Grueiro. Temporary Authority, Permanent Effects: Commit-Time Authorization for LLM Agents. arXiv preprint 2607.10487v1, July 11, 2026. https://arxiv.org/abs/2607.10487v1

[9] V. C. Hu, D. Ferraiolo, R. Kuhn, A. Schnitzer, K. Sandlin, R. Miller, and K. Scarfone. Guide to Attribute Based Access Control (ABAC) Definition and Considerations. NIST SP 800-162, 2014, updated 2019. doi:10.6028/NIST.SP.800-162.

[10] J. Park and R. Sandhu. The UCONABC Usage Control Model. ACM Transactions on Information and System Security, 7(1), 128–174, 2004. doi:10.1145/984334.984339.

[11] OASIS. eXtensible Access Control Markup Language (XACML) Version 3.0. OASIS Standard, January 22, 2013. https://docs.oasis-open.org/xacml/3.0/xacml-3.0-core-spec-os-en.html

[12] H. Wang, C. M. Poskitt, and J. Sun. AgentSpec: Customizable Runtime Enforcement for Safe and Reliable LLM Agents. IEEE/ACM International Conference on Software Engineering, 2026. Institutional publication record: https://ink.library.smu.edu.sg/sis_research/10278/

[13] E. Jaakkola. Designing conceptual articles: four approaches. AMS Review, 10, 18–26, 2020. doi:10.1007/s13162-020-00161-0.

[14] K. Peffers, T. Tuunanen, M. A. Rothenberger, and S. Chatterjee. A Design Science Research Methodology for Information Systems Research. Journal of Management Information Systems, 24(3), 45–77, 2007. doi:10.2753/MIS0742-1222240302.

[15] Amazon Web Services. Revoke IAM role temporary security credentials. IAM User Guide. Accessed September 6, 2026. https://docs.aws.amazon.com/IAM/latest/UserGuide/id_roles_use_revoke-sessions.html

[16] F. T. Liu, K. M. Ting, and Z.-H. Zhou. Isolation Forest. Proceedings of the Eighth IEEE International Conference on Data Mining, 413–422, 2008. doi:10.1109/ICDM.2008.17.

[17] Amazon Web Services. Contain. AWS Security Incident Response User Guide. Accessed September 6, 2026. https://docs.aws.amazon.com/security-ir/latest/userguide/contain.html

[18] Amazon Web Services. CloudTrail record contents for management, data, and network activity events. CloudTrail User Guide. Accessed September 6, 2026. https://docs.aws.amazon.com/awscloudtrail/latest/userguide/cloudtrail-event-reference-record-contents.html

[19] Amazon Web Services. GetRole. IAM API Reference. Accessed September 6, 2026. https://docs.aws.amazon.com/IAM/latest/APIReference/API_GetRole.html

[20] Amazon Web Services. PutRolePolicy. IAM API Reference. Accessed September 6, 2026. https://docs.aws.amazon.com/IAM/latest/APIReference/API_PutRolePolicy.html

[21] Amazon Web Services. IAM and AWS STS quotas. IAM User Guide. Accessed September 6, 2026. https://docs.aws.amazon.com/IAM/latest/UserGuide/reference_iam-quotas.html

[22] Amazon Web Services. Policy evaluation logic. IAM User Guide. Accessed September 6, 2026. https://docs.aws.amazon.com/IAM/latest/UserGuide/reference_policies_evaluation-logic.html

[23] Amazon Web Services. IAM JSON policy elements: Condition operators. IAM User Guide. Accessed September 6, 2026. https://docs.aws.amazon.com/IAM/latest/UserGuide/reference_policies_elements_condition_operators.html

[24] Amazon Web Services. GetRolePolicy. IAM API Reference. Accessed September 6, 2026. https://docs.aws.amazon.com/IAM/latest/APIReference/API_GetRolePolicy.html

[25] Amazon Web Services. Troubleshoot IAM: Changes that I make are not always immediately visible. IAM User Guide. Accessed September 6, 2026. https://docs.aws.amazon.com/IAM/latest/UserGuide/troubleshoot.html

[26] Amazon Web Services. Config Reference: retries and total_max_attempts. Botocore documentation. Accessed September 6, 2026. https://docs.aws.amazon.com/botocore/latest/reference/config.html

[27] Amazon Web Services. DeleteRolePolicy. IAM API Reference. Accessed September 6, 2026. https://docs.aws.amazon.com/IAM/latest/APIReference/API_DeleteRolePolicy.html

[28] Begimher, C. Leo, J. Huang, P. Gaw, and B. Zheng. SIR-Bench: Evaluating Investigation Depth in Security Incident Response Agents. arXiv preprint 2604.12040v1, April 13, 2026. https://arxiv.org/abs/2604.12040v1

Appendix A. Notation and state variables

Symbol or field	Meaning
T, F, ?	True, false, unknown; not probabilities
V, P, E, C, A, B, O	Validity, policy, evidence, context, approval route, budget, readiness
D, R, L	Consumed charge, reserved charge, fixed allowance
w(a)	Nonnegative configured charge for an operation
epoch	Preparation-owner generation checked by the dispatcher
remote	Hidden model state: NONE, QUEUED, APPLIED, FAILED
sends	Number of modeled provider issues for the logical operation
bad_dispatch	Whether issue occurred with invalid modeled approval or context
effect_after_invalidation	Whether a queued effect applied after approval/context changed

The model's consumed JSON field corresponds to D. Its Truth.U enumeration corresponds to ?. Literal phase names are NEW, PREPARED, INTENT, UNCERTAIN, VERIFIED, CANCELLED, and HANDOVER. CHECKED exists only in the split-admission variant.

Appendix B. Reproduction and interpretation

Run python run_all.py from the reproducibility directory. The standard-library runner regenerates results/, including fixtures, gate assignments, composed schedules, mutation checks, graph counts, and counterexample traces. matched_comparison.json contains each controller's twelve-schedule history. The README identifies outputs and source hashes.

Runtime validators check declared fields and bindings; they do not authenticate evidence or implement the full JSON Schema standard. The policy is generated but never submitted to AWS. The finite graph and composed broker remain separate abstractions. Matching hashes identify the tested artifact, not semantic correctness or deployment suitability.

Appendix C. Claim-to-test coverage

Claim	Executed evidence	Boundary not established
Valid-domain gate conformance	2,187 assignments	Truth of admitted evidence

Claim	Executed evidence	Boundary not established
Malformed conditions fail closed	66 negative probes	All possible input encodings
Fixed generated deny contract	Four semantic boundaries; ten structural and two source mutations	Complete IAM semantics
Bound proposal reaches dispatch unchanged	Composed field-substitution and positive traces	Cryptographic signatures and process isolation
Old worker cannot issue after takeover	Two workers call real composed intent/issue methods	Distributed fencing and arbitrary crash schedules
Follow-up survives investigation exhaustion	Per-operation token reservation and consumption	Queue fairness and useful operational sizing
Abstract accounting and issue properties	Eight finite graph configurations	Refinement from graph to broker/database
Comparison with base paper	Attributed reported findings plus separate local schedules	Reproduction of CommitGuard or its benchmark
Residual late effects	Full-model counterexample and three composed schedules	Atomic approval/effect enforcement in AWS